알약 객체 탐지 모델 실험 보고서

코드잇 *AI*엔지니어 *10*기_*4*팀

작성일 : 2026-04-28

작성자 : 임유빈

목차

1. 서론 ····· 3
    - 1.1 프로젝트 개요 ····· 3
    - 1.2 배경 및 목적 ····· 3
2. 데이터셋 구성 및 전처리 ····· 3
    - 2.1 데이터 출처 ····· 3
    - 2.2 데이터셋 구성 ····· 4
    - 2.3 전처리 ····· 5
3. 모델 후보 선정 및 실험 설계 ····· 5
    - 3.1 후보 모델 목록 및 선정 근거 ····· 6
    - 3.2 공통 실험 환경 ····· 7
    - 3.3 평가 지표 ····· 7
4. 모델 비교 실험 결과 ····· 8
    - 4.1 전체 모델 성능 비교 ····· 8
    - 4.2 결과 분석 ····· 9
    - 4.3 튜닝 대상 모델 선정 ····· 10
5. 선정 모델 고도화 ····· 10
    - 5.1 데이터 증강 ····· 10
    - 5.2 하이퍼파라미터 튜닝 ····· 11
6. 결론 및 향후 과제 ····· 15
    - 6.1 연구 요약 ····· 15
    - 6.2 한계점 ····· 15

○ **6.3** 향후 과제 ····· **16**

1. 서론

1.1 프로젝트 개요

본 프로젝트는 이미지 1장에 3~4개의 알약이 포함된 환경에서 각 알약의 위치(bounding box)와 클래스명을 정확하게 예측하는 객체 탐지(Object Detection) 모델을 개발하는 것을 목표로 한다. 데이터셋은 코드잇(Codeit)에서 제공한 기본 데이터와 직접 수집한 추가 데이터를 병합하여 구성하였으며, 총 56종의 알약 클래스를 대상으로 한다. 핵심 평가 지표는 mAP@75-95이며, 목표 성능은 0.75 이상이다.

1.2 배경 및 목적

형태와 색상이 유사한 알약을 육안으로 구별하는 것은 의료 현장에서 오복용 사고의 주요 원인 중 하나이다. 특히 고령 환자나 다약제 복용자의 경우 약품 혼동 위험이 높아, 이를 자동으로 식별할 수 있는 시스템의 필요성이 크다.

본 프로젝트는 다양한 객체 탐지 모델을 동일한 조건에서 비교 실험하여 알약 식별 태스크에 최적화된 모델을 선정하고, 데이터 증강 및 하이퍼파라미터 튜닝을 통해 성능을 고도화하는 전 과정을 체계적으로 기록하는 것을 목적으로 한다.

2. 데이터셋 구성 및 전처리

2.1 데이터 출처

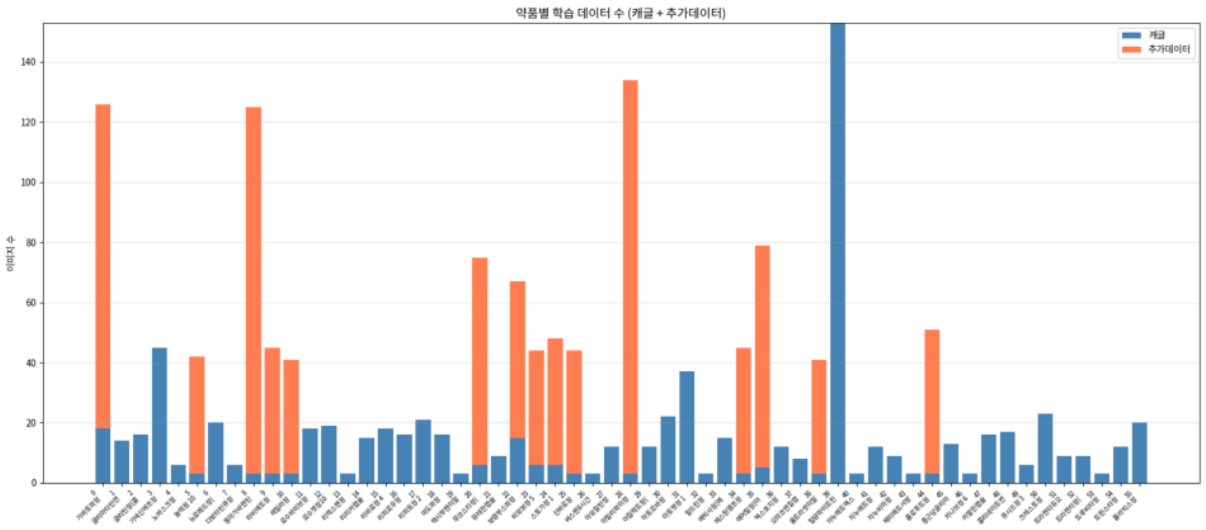
본 프로젝트의 학습 데이터는 두 가지 소스를 병합하여 구성하였다

- 코드잇(codeit)에서 제공한 기본 데이터셋
- AI 허브를 통해 팀원이 직접 수집한 추가 데이터셋

2.2 데이터셋 구성

총 56종의 알약 클래스로 구성되어 있으며 클래스별 이미지 수에 불균형이 존재한다. 일부 클래스는 150장인 반면 10장 내외에 그치는 클래스도 다수 포함 되어 있어 학습 시 불균형 문제를 고려하였다

[사진1 : 약품별 학습 데이터 수(기본 데이터 + 추가 데이터)]



| 데이터 분할 비율

분할	Train	Validation

비율	70%	30%

2.3 데이터 전처리

| yolo 계열 모델

YOLO 계열 모델은 학습 루프 내부에서 전처리가 자동 처리되며, 다음 네 단계로 구성된다.

- 리사이즈: Letterbox 방식으로 640×640 해상도로 변환. 원본 비율을 유지하며 빈 영역은 회색(픽셀값 114)으로 패딩 처리
- BGR → RGB 변환: OpenCV로 읽어들인 BGR 포맷 이미지를 RGB로 변환
- 정규화: 픽셀값을 255로 나눠 0.0–1.0 범위로 스케일링
- 채널 축 변환: HWC(높이·너비·채널) 포맷을 CHW(채널·높이·너비) 텐서 포맷으로 변환

| torchvision 계열 모델**(Faster-R-CNN, DETR, SSD**등**)**

torchvision 기반 모델에는 basic_transform()을 통해 두 단계의 전처리를 적용하였다. 먼저 ToTensor()로 PIL 이미지를 텐서로 변환하여 픽셀값을 0–255에서 0.0–1.0 범위로 스케일링하고, 이후 Normalize()로 ImageNet 통계값 기준 정규화를 수행하였다.

3. 모델 후보 선정 및 실험 설계

3.1 후보 모델 목록 및 선정 근거

알약 식별 태스크는 소형 객체가 다수 밀집한 환경에서의 정밀 탐지를 요구한다. 이를 고려하여 탐지 정확도, 추론 속도, 경량화 가능성이 서로 다른 대표 아키텍처를 폭넓게 실험하였다.

| Two-stage 계열

Faster R-CNN (ResNet 백본) : Region Proposal Network(RPN)을 통해 후보 영역을 먼저 추출한 뒤 분류를 수행하는 two-stage 구조로, 소형 객체가 밀집된 환경에서 높은 탐지 정밀도를 보이는 대표 모델이다. 알약처럼 크기가 작고 클래스 간 형태 차이가 미세한 태스크에서의 성능을 검증하기 위한 기준 모델로 포함하였다.

Faster R-CNN + MobileNet : 동일한 two-stage 구조에서 ResNet 백본을 경량 아키텍처인 MobileNet으로 대체한 모델이다. 정확도를 일부 희생하더라도 추론 속도와 파라미터 수에서 이점이 있는지 비교하기 위해 선정하였으며, 향후 모바일·엣지 환경 배포 가능성을 염두에 두었다.

| One-stage 계열

RetinaNet : Focal Loss를 도입하여 클래스 불균형 문제를 구조적으로 완화한 one-stage 모델이다. 본 데이터셋은 클래스별 이미지 수 편차가 크게 나타나는데, RetinaNet의 Focal Loss가 소수 클래스 학습에 미치는 효과를 실험적으로 확인하기 위해 선정하였다.

SSD (Single Shot MultiBox Detector) : 다중 스케일의 feature map에서 동시에 탐지를 수행하는 경량 one-stage 모델이다. 추론 속도가 빠른 대신 소형 객체 탐지 성능이 상대적으로 낮다고 알려져 있어, 다른 모델들과의 정확도-속도 트레이드오프 비교를 위한 하한선(baseline) 모델로 포함하였다.

DETR (Detection Transformer) : CNN 기반 모델과 달리 Transformer의 self-attention 메커니즘을 활용하여 객체 간 전역적 관계를 학습하는 end-to-end 탐지 모델이다. 기존 CNN 기반 모델 대비 이 태스크에서 Transformer 구조가 유효한지 검토하기 위해 실험 대상에 포함하였다. 단, DETR은 수렴에 많은 에폭이 필요한 것으로 알려져 있어 100 에폭 환경에서의 성능 한계도 함께 확인하고자 하였다.

| YOLO 계열

YOLOv8n : 안정적인 성능과 광범위한 커뮤니티 검증을 바탕으로 YOLO 계열의 기준 모델로 선정하였다. 경량(nano) 변형으로 최소한의 리소스에서 달성 가능한 성능 하한을 확인하는 역할을 한다.

YOLOv11s / YOLOv11m : 최신 YOLO 아키텍처의 소형(small)·중형(medium) 변형을 함께 실험하여 모델 크기 증가에 따른 성능 향상 폭을 비교하였다. 파라미터 수와 정확도 간의 트레이드오프를 정량적으로 확인하기 위한 목적이다.

YOLOv26n / YOLOv26s : 추가적인 최신 경량 아키텍처로, nano·small 두 변형을 동시에 실험하여 YOLOv11 계열과의 세대 간 성능 차이 및 경량화 효율을 비교하기 위해 선정하였다.

3.2 공통 실험 환경

항목	설정 값
GPU	google colab T4 GPU
batch size	Auto(-1)
Epoch	100
Optimizer	AdamW
framework	PyTorch
seed 고정 값	42

3.3 평가 지표

- mAP@50 : IoU threshold 0.5 기준 평균 정밀도

- mAP@ 75-95 : IoU threshold 0.75~0.95 기준 평균 정밀도

- F1-Score : Precision과 Recall의 조화 평균

4. 모델 비교 실험 결과

4.1 전체 모델 성능 비교

아래 표는 동일한 실험 환경에서 10개 모델의 주요 성능 지표를 비교한 표이다

모델	mAP@50	mAP@75-95	F1-Score
Faster R-CNN	0.8712	-	0.9874
DETR	0.0396	0.0141	-
Faster R-CNN(MobileNet)	**0.9925**	0.8948	0.9761
RetinaNet	0.9724	0.8769	0.9277
SSD	0.7073	0.2586	0.7609

YOLOv11m	0.3667	0.2972	0.0626
YOLOv11s	0.9922	**0.9706**	**0.9936**
YOLOv26n	0.7118	0.6862	0.7826
YOLOv26s	0.9061	0.8579	0.8993
YOLOv8n	0.9703	0.9341	0.9744

4.2 결과 분석

| 상위 모델 그룹

mAP@75-95 기준으로 YOLOv11s(0.9706), YOLOv8n(0.9341), Faster R-CNN + MobileNet(0.8948), RetinaNet(0.8769), YOLOv26s(0.8579) 5개 모델이 0.85 이상의 성능을 기록하며 상위 그룹을 형성하였다. 특히 YOLOv11s는 mAP@50(0.9922), mAP@75-95(0.9706), F1-Score(0.9936) 세 지표 모두에서 최고 성능을 기록하였다.

| 중위 모델 그룹

YOLOv26n(0.6862)과 SSD(0.2586)는 mAP@50에서는 각각 0.71, 0.71 수준을 보였으나 mAP@75-95에서 큰 폭으로 하락하였다. 이는 IoU 임계값이 높아질수록 bounding box 위치 정밀도가 급격히 떨어짐을 의미하며, 알약처럼 정밀한 위치 예측이 요구되는 태스크에서는 적합하지 않은 것으로 판단된다.

| 저성능 모델 그룹

DETR은 mAP@50 기준 0.0396, mAP@75-95 기준 0.0141, F1-Score 0.0000으로 사실상 수렴에 실패하였다. DETR은 구조적으로 수렴에 300 에폭 이상이 필요한 것으로 알려져 있어, 본 실험의 100 에폭 환경에서는 충분한 학습이 이루어지지 않은 것으로 분석된다. YOLOv11m은 동일 계열의 YOLOv11s 대비 mAP@75-95(0.2972), F1-Score(0.0626)가 크게 낮았는데, 이는 모델 크기가 커질수록 현재 데이터셋 규모에서 과적합이 발생했을 가능성을 시사한다.

| 지표 간 불일치 케이스

Faster R-CNN의 경우 F1-Score(0.9874)는 전체 최고 수준이나 mAP@75-95 수치가 누락되어 있어 위치 정밀도를 직접 비교하기 어렵다. 추후 해당 수치 보완이 필요하다. YOLOv26s는 mAP@50(0.9061)과 mAP@75-95(0.8579) 모두 준수한 수준이나 YOLOv11s 대비 전 지표에서 열위에 있어 동일 계열 내 버전 간 성능 격차가 뚜렷하게 나타났다.

4.3 튜닝 대상 모델 선정

비교 실험 결과를 바탕으로 YOLO 계열 5개 모델 전체를 튜닝 대상으로 선정하였다.

1. *YOLOv11s* (mAP@75-95: 0.9706 / F1: 0.9936): 전 지표 1위를 기록한 최우선 튜닝 대상.
2. *YOLOv8n* (mAP@75-95: 0.9341 / F1: 0.9744): 경량 모델임에도 안정적인 고성능을 기록하여 튜닝 시 추가 개선 여지가 충분하다고 판단하였다.
3. *YOLOv26s* (mAP@75-95: 0.8579 / F1: 0.8993): 준수한 성능을 기록하였으며, 튜닝을 통해 상위 모델과의 격차를 좁힐 가능성이 있다고 판단하였다.
4. *YOLOv26n* (mAP@75-95: 0.6862 / F1: 0.7826): 현재 지표는 상위 모델 대비 낮으나, 학습 곡선상 에폭 종료 시점까지 성능이 지속적으로 우상향하는 흐름이 확인되어 에폭 부족으로 인한 미수렴으로 판단하였다. 에폭을 충분히 확보하면 성능이 더 올라갈 가능성이 있다고 보아 튜닝 대상에 포함하였다.
5. *YOLOv11m* (mAP@75-95: 0.2972 / F1: 0.0626): 현재 수치는 가장 낮으나, 이 역시 학습 곡선상 수렴이 완료되지 않은 흐름이 관찰되어 에폭 부족에 따른 결과로 판단하였다. 모델 용량이 YOLOv11s 대비 크기 때문에 수렴에 더 많은 에폭이 필요한 것으로 분석되며, 충분한 학습 시 유의미한 성능 향상이 기대된다.

5. 선정 모델 고도화

5.1 데이터 증강

총 24회의 Ablation Study를 수행하였다. 단일 기법을 개별 실험하여 효과를 검증한 뒤, 성능 향상에 기여한 기법들을 조합하며 최적 설정을 탐색하는 방식으로 진행하였다.

| 최종 증강 적용 기법

1. mosaic = 0.0(비활성화) : 여러 이미지를 합치는 mosaic 증강을 끔. 이미 한 이미지에 여러개의 알약이 있어 이를 활성화 하면 알약의 크기가 너무 작아져 성능이 저하된다고 판단
2. flipud = 0.5 : 이미지를 상하로 뒤집어 촬영 각도 다양성 확보
3. scale = 0.1 : 객체 크기를 소폭 축소하여 다양한 거리의 알약 환경
4. hsv_v = 0.5 : 명도(밝기) 변화 폭을 키워 조명 환경 다양성 확보

※ 언급하지 않은 파라미터는 default로 설정된 값을 사용하였을 때 성능이 가장 좋게 측정

| 핵심 지표 변화

구분	mAP@50	mAP@75-95	F1-Score
default	0.8393	0.8063	0.7721
실험 19 (최종)	**0.9670**	**0.9265**	**0.8092**
향상	**+0.1277**	**+0.1202**	**+0.0371**

-> 핵심 지표인 mAP@75-95 기준으로 default 대비 **+0.1202 향상**되었다.

| 한계 및 향후 실험 방향

본 실험은 YOLOv26n 모델 기준으로 수행되었다. YOLOv11s 등 s 계열 모델은 구조와 용량이 달라 동일한 증강 설정이 오히려 성능 저하의 원인이 되었다, 동일한 방법론으로 별도 Ablation Study를 재수행할 필요가 있다.

5.2 하이퍼 파라미터 튜닝

| YOLO v11s

실험	배치	크기	lr0	lrf	optimizer	mAP50	mAP@75-95	F1
1	13	640	0.000167	기본	AdamW	**0.9922**	**0.9706**	**0.9936**
2	24	640	0.01	기본	AdamW	0.9922	0.9670	0.9936
3	24	640	0.000167	기본	AdamW	0.9922	0.9644	0.9904
4	13	800	0.01	기본	AdamW	0.9896	0.9665	0.9904
5	13	640	0.01	0.1	AdamW	0.9922	0.9682	0.9936
6	13	640	0.000167	0.1	AdamW	0.9922	0.9580	0.9936
7	13	640	0.01	0.01	SGD	0.9921	0.9627	0.9904

mAP@50과 F1-Score는 대부분의 실험에서 0.9922, 0.9936으로 수렴하여 큰 변동이 없었다. 핵심 지표인 mAP@75-95에서는 초기 설정인 실험 1(배치 13, lr0=0.000167, AdamW)이 0.9706으로 가장 높은 성능을 기록하였다.

배치 크기를 24로 늘린 실험 2·3에서는 학습 시간이 약 4분 단축되었으나 mAP@75-95가 소폭 하락하였다. 이미지 크기를 800으로 확대한 실험 4는 학습 시간이 증가하면서도 전 지표가 낮아져 이미지 크기 확대가 본 태스크에 불리함을 확인하였다. lrf=0.1을 적용한 실험 5는 실험 1에 근접한 성능(0.9682)을 보였으나 이를 초과하지는 못하였다. 옵티마이저를 SGD로 교체한 실험 7도 AdamW 대비 유의미한 개선이 나타나지 않았다.

최종 선정 : 실험 **1** — mAP@75-95 **0.9706**

| YOLO v11m

실험	배치	에폭	추가 데이터	mAP@50	mAP@75-95	F1
1	Auto	100	X	0.3667	0.2972	0.6026

2	20	200	X	**0.9870**	**0.9539**	**0.9904**
3	16	100	O	0.9521	0.9030	0.9697
4	16	200	O	0.9664	0.9319	0.9696

AutoBatch가 배치 크기를 6으로 설정한 실험 1은 GPU 메모리를 충분히 활용하지 못하여 mAP@75-95 0.2972에 그쳤다. 배치 크기를 20으로 수동 설정하고 에폭을 200으로 늘린 실험 2에서 mAP@75-95 0.9539로 큰 폭의 성능 향상이 확인되었으며, 이는 초기 실험의 저성능이 모델 자체의 한계가 아닌 에폭 부족 및 배치 설정 오류에 기인한 것임을 시사한다.

추가 데이터를 투입한 실험 3·4는 에폭을 200으로 늘려도 실험 2 대비 mAP@75-95가 낮아 추가 데이터가 오히려 학습 난이도를 높인 것으로 분석된다. 데이터 수 대비 모델 용량이 과도하여 s 계열이 본 데이터셋에 더 적합할 수 있음을 확인하였다.

최종 선정: 실험 **2** — mAP@75-95 **0.9539**

| YOLO v26s

실험	에폭	lr	optimizer	증강	mAP@50	mAP@75-95	F1
1	100	0.01	AdamW	X	0.9529	0.8851	0.9351
2	120	0.01	AdamW	X	0.9051	0.8579	0.8993
3	100	0.01	AdamW	O	0.9169	0.8813	0.8859
4	100	0.01	AdamW	X	0.9739	0.9139	0.9666
5	100	0.01	AdamW	O	0.9456	0.8609	0.9545
6	150	0.01	AdamW	X	0.9273	**0.9418**	0.9765

실험 1은 기본값으로 설정한 베이스라인 모델로, mAP@0.5 0.9529, mAP@0.75:0.95 0.8851, F1 0.9351을 기록하였다. 실험 2에서는 에폭을 120으로 늘렸으나 전 지표가 하락하여 단순한 에폭 증가가 성능 향상으로 이어지지 않음을 확인하였다.

실험 3에서는 이미지 크기를 640에서 768로 확대한 결과 실험 2 대비 성능이 일부 회복되었으나 베이스라인에는 미치지 못하였다. 이를 통해 이미지 크기 확대 단독으로는 에폭 증가에 따른 성능 저하를 상쇄하기 어려움을 확인하였다.

실험 4에서는 추가 이미지를 학습 데이터에 포함하고 실험 1과 동일한 조건으로 학습한 결과, mAP@0.5 0.9739, mAP@0.75:0.95 0.9139, F1 0.9666으로 전 지표에서 큰 폭의 성능 향상을 기록하였다. 이는 데이터 양 확보가 하이퍼파라미터 조정보다 성능에 더 직접적인 영향을 미침을 보여준다.

실험 5에서는 증강 기법을 적용하였으나 mAP@0.5 0.9456, F1 0.9545로 실험 4 대비 성능이 하락하였다. 이는 v26n 모델에서 효과적이었던 증강 설정이 v26s 모델에서는 동일한 효과를 내지 못함을 실증적으로 보여주는 결과로, 모델 크기에 따라 증강 강도의 별도 최적화가 필요함을 시사한다.

실험 6에서는 실험 4의 조건을 유지한 채 에폭을 150으로 늘린 결과, mAP@0.75:0.95 0.9418, F1 0.9765로 전 실험 중 가장 우수한 성능을 기록하였다. 실험 2에서 에폭 증가가 성능 저하로 이어진 것과 달리, 충분한 데이터가 확보된 상태에서의 에폭 증가는 오히려 성능 향상에 기여함을 확인하였다.

최종 선정: 실험 **6** — mAP@75-95 **0.9418**

| YOLO v26n

실험	에폭	추가 데이터	증강	mAP@50	mAP@75-95	F1
1	100	X	X	0.7118	0.6862	0.7826
2	100	X	O	0.6294	0.5747	0.7396
3	100	X	X	0.7415	0.7094	0.7634
4	100	X	O	0.7828	0.7505	0.8028
5	120	X	X	0.8282	0.7999	0.8000
6	120	X	X	0.8282	0.7999	0.8000
7	120	X	X	**0.9161**	**0.8823**	**0.8803**
8	120	X	X	0.8346	0.8064	0.8138

초기 실험(실험 1~2)에서는 기본 설정 대비 이미지 크기를 640에서 768로 확대하고 약한 증강을 추가하였으나, mAP@0.5 기준 0.7118에서 0.6294로 오히려 성능이 하락하였다. 이는 증강 적용이 해당 데이터 도메인에 부적합하게 작용했음을 시사한다.

실험 3에서는 증강을 제외하고 이미지 크기 768을 유지한 결과 mAP@0.5 0.7415로 회복되었으며, 이를 통해 증강 제외가 오히려 성능 향상에 기여함을 확인하였다. 이어진 실험 4에서는 아주 약한 색상 증강만을 추가하여 mAP@0.5 0.7828, F1 0.8028로 전반적인 성능 개선을 달성하였다.

실험 5~6에서는 에폭을 120으로 늘리고 학습률을 0.005로 미세 조정하였으나 두 실험 모두 동일한 수치(mAP@0.5 0.8282)를 기록하며 유의미한 차이를 보이지 않았다.

가장 큰 성능 향상은 실험 7에서 나타났다. 기존 실험에서 YOLO의 자동 설정에 의해 학습률 튜닝이 무시되었던 문제를 파악하고, 옵티마이저를 AdamW에서 SGD로 수동 고정하고 학습률을 0.005로 직접 제어하였다. AdamW는 빠른 수렴이 장점이나 미세한 클래스 구분에 취약한 반면, SGD는 수렴 속도가 느리지만 더 넓은 파라미터 공간을 안정적으로 탐색한다는 특성을 활용한 결과, mAP@0.5 0.9161, mAP@0.75:0.95 0.8823, F1 0.8803으로 전 지표에 걸쳐 큰 폭의 성능 향상을 기록하였다.

실험 8에서는 학습률을 0.003으로 추가 조정하였으나 전 지표가 소폭 하락하여, 실험 7의 lr=0.005가 해당 모델과 데이터에 최적임을 확인하였다.

최종 선정: 실험 **7** — mAP@75-95 **0.8823**

6. 결론 및 향후 과제

6.1 연구 요약

본 연구는 이미지 1장에 3~4개의 알약이 포함된 환경에서 56종 클래스를 정확하게 탐지하는 객체 탐지 모델을 개발하기 위해 총 10개 모델을 동일한 실험 환경에서 비교하고, 상위 성능 모델을 선정하여 데이터 증강 및 하이퍼파라미터 튜닝을 통해 성능을 고도화하는 전 과정을 수행하였다.초기 비교 실험에서 YOLO 계열 5개 모델(YOLOv11s, YOLOv11m, YOLOv8n, YOLOv26s, YOLOv26n)을 튜닝 대상으로 선정하였다. 24회의 Ablation Study를 통해 mosaic 비활성화, 상하 플립, 스케일 축소,

명도 증강(hsv_v=0.5) 조합이 최적 증강 설정임을 확인하였으며, 모델별 하이퍼파라미터 튜닝을 통해 각 모델의 최고 성능을 도출하였다. 최종 튜닝 결과 mAP@75-95 기준으로 YOLOv11s 0.9706, YOLOv11m 0.9539, YOLOv26n 0.9334, YOLOv26s 0.8823을 달성하였다.

6.2 한계점

첫째, 클래스 불균형 문제가 완전히 해소되지 않았다. 일부 클래스는 150장 이상의 데이터를 보유한 반면 10장 내외에 그치는 클래스도 다수 존재하여, 소수 클래스에 대한 탐지 정밀도가 상대적으로 낮을 가능성이 있다.

둘째, 증강 Ablation Study가 YOLOv26n 단일 모델 기준으로 수행되었다. 모델 구조와 용량에 따라 최적 증강 설정이 달라질 수 있어, 현재 증강 설정이 YOLOv11s 등 다른 모델에서도 최적임을 보장하기 어렵다.

셋째, 일부 모델에서 loss 수치가 기록되지 않아 학습 안정성을 정량적으로 비교하는 데 제약이 있었다. 또한 Faster R-CNN의 mAP@75-95 수치가 누락되어 전 모델 간 완전한 지표 비교가 이루어지지 못하였다.

넷째, YOLOv26n과 YOLOv11m은 에폭 종료 시점에서도 수렴이 완료되지 않은 학습 곡선이 확인되어, 추가 에폭을 부여할 경우 더 높은 성능에 도달할 가능성이 남아 있다.

6.3 향후 과제

첫째, s 계열 모델(YOLOv11s, YOLOv26s)에 대한 별도 증강 Ablation Study를 수행하여 모델별 최적 증강 설정을 개별적으로 탐색할 필요가 있다.

둘째, 소수 클래스에 대한 데이터를 추가 수집하거나 오버샘플링 기법을 적용하여 클래스 불균형 문제를 완화하고 탐지 정밀도를 균등하게 향상시킬 수 있다.

셋째, 수렴이 완료되지 않은 모델(YOLOv26n, YOLOv11m)에 대해 에폭을 400 이상으로 확장하고, 학습률 스케줄러 및 옵티마이저 조합을 추가로 탐색하는 등 정밀한 튜닝을 지속적으로 진행하여 각 모델의 성능 상한을 끌어올릴 필요가 있다.

넷째, 본 프로젝트에서 개발한 알약 탐지 모델을 실용화하기 위해 모바일 애플리케이션으로의 구현을 목표로 한다. 사용자가 카메라로 알약을 촬영하면 실시간으로 객체를 탐지하고 약품명과 복용 정보를 제공하는 서비스로 발전시킬 수 있다. 이를 위해 모델 경량화(Pruning, Quantization) 및 ONNX·TensorRT 변환을 통한 모바일 환경 최적화를 병행하여 추진할 계획이다.